PolyVote: A Declarative, Verifiable E-Voting Architecture

PIBA – Software Architecture Lab, WS 2026/27

Tokyo • David Enoghama, Franka Heger, Clemens Korneli, Patrick Ng, Felix Roehr, Jerome Stephan

Problem

Organizations running digital elections (university committees, student unions, associations) face a trade-off between **ballot flexibility** and **cryptographic verifiability**. Practical e-voting systems (*Helios*, *Belenios*) enforce rigid formats supporting only 1-of-N or approval voting, while flexible research frameworks (*Kryvos*, *Ordinos*) lack usable implementations. Survey tools like *LimeSurvey* let officials build custom GUI ballots, but lack verifiability and require blind trust in a central admin.

Idea

We propose **PolyVote**, an e-voting architecture combining a **No-Code Election Studio GUI** and a **Declarative Election-as-Code Compiler Pipeline** around an encapsulated General-Purpose Zero-Knowledge Proof (GPZKP) adapter. Officials visually configure custom voting schemes (e.g., Ranked-Choice, Quadratic, or Cumulative Voting) in the Studio GUI, emitting a declarative JSON/DSL specification. Our Compiler Pipeline translates this specification into three synchronized artifacts: (1) a dynamic *Voter Ballot UI* schema, (2) constraints for an abstract *GPZKP Prover/Verifier Adapter*, and (3) accumulation rules for the *Voting Authority Service*. Voters compute encrypted ballots and GPZKP validity proofs client-side and post them to an append-only *Bulletin Board* for verification.
Quality Attributes & Scope: Driven by *Modifiability & Configurability* (new voting modes without code changes), *Decoupling* (isolating the black-box GPZKP primitive behind an Adapter), and *Auditability. Out of Scope:* Cryptographic performance tuning, new ZKP primitives, Benaloh challenges.

Benefits

- **No-Code Ballot Expressiveness** — addresses the rigid formats of *Helios*/*Belenios* and missing implementations of *Kryvos*/*Ordinos*; demonstrated by configuring and compiling Plurality, Ranked-Choice, and Quadratic Voting in the GUI without code changes.
- **Survey Usability with Public Auditability** — addresses *LimeSurvey*'s black-box trust model; demonstrated by posting encrypted ballots and validity proofs to the Bulletin Board.
- **Cryptographic Agnosticism & Isolation** — addresses tight coupling to experimental cryptography; demonstrated by swapping the GPZKP adapter between a mock and the real primitive.

Action

1. **Requirements & Domain Models (27.11. & 11.12.2026):** Elicit requirements (27.11.) and model the domain and declarative *Election-as-Code* grammar for ballots and budgets (11.12.).
2. **Architecture Design (08.01.2027):** Specify component boundaries, the three-target Compiler Pipeline, Bulletin Board protocol, and Voting Authority verification workflow.
3. **Prototype & Evaluation (22.01.–05.02.2027):** Build the *Minimum Deliverable* (Studio GUI, Compiler, Voter Client, Bulletin Board, Voting Authority) and evaluate modifiability.

Tech & Team Split (6): Java 21/Spring Boot, TypeScript, PostgreSQL, Docker: (1) *Studio GUI*, (2) *DSL/Validator*, (3) *Compiler*, (4) *Voter/Prover Adapter*, (5) *Bulletin Board*, (6) *Voting Authority*.

Prior work disclaimer. One member worked on GPZKP ballot-validity primitives in their Bachelor's thesis; to avoid conflict of interest, it is used strictly as a black-box dependency without runtime guarantees.
AI usage statement. Google Gemini was used to brainstorm the architecture and condense the PIBA text.